Computational Logic in the Era of AI

Numbers, Induction, and Inductive Types

Sergey Mechtaev

mechtaev@pku.edu.cn

Term 1 • 26–27

This lecture



1 Peano axioms

ℕ, 0, s as primitives; Church numerals

—

► Postulates do not compute; codings give no induction.

2 Recursors

constructors, recursor, ι-reduction

`inductive, Nat.rec`

► $fac\,3 = 6$ by computation.

3 Induction

the recursor at a `Prop` motive

`induction, cases`

► Induction is a rule, not an axiom.

4 Inductive predicates

rules as constructors; $\wedge$, $\vee$, $=$ as inductives

`Even, Eq`

► A relation defined by rules has its own induction.

5 Automation

termination; decision procedures

`omega, grind`

► What does not compute needs a measure or a procedure.

History



Giuseppe Peano

1858–1932

Zero, successor, and induction, as axioms.

1889 Peano: a set $\mathbb{N}$, an element 0, a successor s, and axioms, induction among them.

1936 Church, Turing: no machine decides whether a formula is derivable.

1972 Girard: impredicativity, so Church numerals, $\wedge$ and $\vee$ become definable inside the type system.

1980 Martin-Löf, lectures at Padova: inductive types as primitives, so proofs by induction and definitions by recursion are built in.

1988 Coquand and Huet: the Calculus of Constructions, λC; extended with inductive types it becomes CIC, the logic of Coq.



Per Martin-Löf

born 1942

Induction and recursion as rules of the type theory.

Photos: Wikimedia; A. Bauer (CC BY-SA 2.5 si), via Wikimedia Commons.

N&G §5.7, §6.5, §14.15, §16.5

Peano's numbers in λD

The numbers are six primitive definitions.

(1)	$\mathbb{N} := \amalg : *_s$	the set
(2)	$0 := \amalg : \mathbb{N}$	zero
(3)	$s := \amalg : \mathbb{N} \to \mathbb{N}$	successor
(4)	$ax\text{-}nat_1 := \amalg : \forall x{:}\mathbb{N}.\ \neg(s\,x =_{\mathbb{N}} 0)$	s never gives 0
(5)	$ax\text{-}nat_2 := \amalg : \forall x, y{:}\mathbb{N}.\ (s\,x =_{\mathbb{N}} s\,y \Rightarrow x =_{\mathbb{N}} y)$	s is injective
(a)	$P : \mathbb{N} \to *_p$	
(6)	$ax\text{-}nat_3(P) := \amalg : (P\,0 \wedge \forall x{:}\mathbb{N}.\ (P\,x \Rightarrow P\,(s\,x))) \Rightarrow \forall x{:}\mathbb{N}.\ P\,x$	induction

N&G §14.1, Fig. 14.1

Induction, used

Lemma. Every number is 0 or a successor. Apply $ax\text{-}nat_3$ to $P \equiv \lambda n{:}\mathbb{N}.\ (n =_{\mathbb{N}} 0 \vee \exists m{:}\mathbb{N}.\ n =_{\mathbb{N}} s\,m)$.

(1)	$P\,0$	$\vee$-in$_1$, since $0 =_{\mathbb{N}} 0$
(*b*)	$k : \mathbb{N}$	
(*c*)	$u : P\,k$	
(2)	$\exists m{:}\mathbb{N}.\ s\,k =_{\mathbb{N}} s\,m$	$\exists$-in, witness k
(3)	$P\,(s\,k)$	$\vee$-in$_2$; u is not used
(4)	$\forall k{:}\mathbb{N}.\ (P\,k \Rightarrow P\,(s\,k))$	$\Rightarrow$-in, $\forall$-in
(5)	$\forall n{:}\mathbb{N}.\ P\,n$	$ax\text{-}nat_3(P)$ on (1), (4)

N&G §14.1, Lem. 14.1.4

$fac\,3 = 6$, **proved in** λD

A recursive function and its equations are primitive definitions:

$$fac := \perp\!\!\!\perp : \mathbb{N}\to\mathbb{N} \qquad fac\text{-}0 := \perp\!\!\!\perp : fac\,0 = 1 \qquad fac\text{-}s := \perp\!\!\!\perp : \forall n{:}\mathbb{N}.\, fac\,(s\,n) = s\,n \cdot fac\,n$$

(1)	$a_1 := fac\text{-}s\ 2 \ : \ fac\,3 = 3 \cdot fac\,2$	∀-el; 3 is $s\,2$
(2)	$a_2 := fac\text{-}s\ 1 \ : \ fac\,2 = 2 \cdot fac\,1$	∀-el
(3)	$a_3 := fac\text{-}s\ 0 \ : \ fac\,1 = 1 \cdot fac\,0$	∀-el
(4)	$a_4 := eq\text{-}trans(a_3,\ eq\text{-}cong(1 \cdot _,\ fac\text{-}0)) \ : \ fac\,1 = 1 \cdot 1$	fac-0 inside $1 \cdot _$
(5)	$a_5 := eq\text{-}trans(a_2,\ eq\text{-}cong(2 \cdot _,\ a_4)) \ : \ fac\,2 = 2 \cdot (1 \cdot 1)$	(4) inside $2 \cdot _$
(6)	$a_6 := eq\text{-}trans(a_1,\ eq\text{-}cong(3 \cdot _,\ a_5)) \ : \ fac\,3 = 3 \cdot (2 \cdot (1 \cdot 1))$	(5) inside $3 \cdot _$
(7)	$a_7 := \ldots \ : \ 3 \cdot (2 \cdot (1 \cdot 1)) = 6$	equations of $\cdot$
(8)	$a_8 := eq\text{-}trans(a_6,\ a_7) \ : \ fac\,3 = 6$	

Each line instantiates an axiom, substitutes, or chains two equalities. N&G §8.2

Proving $fac\,3 = 6$, or computing it

Suppose instead that fac had a body, and the body reduced.

in λD: prove

$$
\begin{aligned}
fac\,3 &= 3 \cdot fac\,2 && fac\text{-}s\\
&= 3 \cdot 2 \cdot fac\,1 && fac\text{-}s\\
&= 3 \cdot 2 \cdot 1 \cdot fac\,0 && fac\text{-}s\\
&= 6 && fac\text{-}0
\end{aligned}
$$

each $=$ is lines of a derivation:
the proof grows with n

wanted: compute

$$
\begin{aligned}
fac\,3 &\twoheadrightarrow 3 \cdot fac\,2\\
&\twoheadrightarrow 3 \cdot 2 \cdot fac\,1\\
&\twoheadrightarrow 3 \cdot 2 \cdot 1 \cdot fac\,0\\
&\twoheadrightarrow 6
\end{aligned}
$$

each $\twoheadrightarrow$ is a reduction:
(*conv*) takes it silently

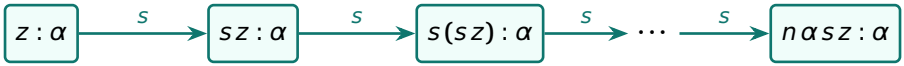
On the right the proof of $fac\,3 = 6$ is $eq\text{-}refl(\mathbb{N}, 6)$.

N&G §8.2, §16.2

Church numerals in $\lambda 2$

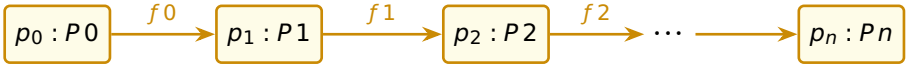
A numeral n applies a function n times: $\mathbb{N}_2 \equiv \Pi\alpha{:}\,*\,.\,(\alpha \to \alpha) \to \alpha \to \alpha$.

what a numeral can do: iterate



one type α at every step, chosen before n is known

what induction needs



a different type at every step: $P\,k$ depends on k, $f : \Pi k{:}\mathbb{N}_2.\,P\,k \to P\,(s\,k)$

α cannot mention n, so $\forall n{:}\mathbb{N}_2.\,P\,n$ cannot be derived.

N&G §6.5

Numbers: computation and induction

numbers as	computation	induction
primitive definitions (λD)	×	✓
Church numerals ($\lambda 2$)	✓	×
an inductive type	✓	✓

An **inductive type** is a type given by its constructors, here 0 and s. N&G §6.5

An inductive type is four rules

rule	**for $\mathbb{N}$**	**says**
formation	$\mathbb{N} : *_s$	the type exists
introduction	$0 : \mathbb{N}, \quad s : \mathbb{N} \to \mathbb{N}$	how to build one; nothing else is one
elimination	$rec : \Pi P{:}\mathbb{N} \to *_. \, P\,0 \to (\Pi n{:}\mathbb{N}_. \, P\,n \to P\,(s\,n)) \to \Pi n{:}\mathbb{N}_. \, P\,n$	how to use one: a case per constructor
computation	$rec\,P\,z\,f\,0 \to_\iota z, \quad rec\,P\,z\,f\,(s\,n) \to_\iota f\,n\,(rec\,P\,z\,f\,n)$	the recursor runs

The **recursor** rec produces, for every n, an inhabitant of $P\,n$. P is its **motive**, and $*$ may be $*_s$ or $*_p$.

N&G §6.5, §16.2; TPIL §7.8

The rule (*elim*) for $\mathbb{N}$

$$\frac{\Gamma \vdash P : \mathbb{N} \to * \quad \Gamma \vdash z : P\,0 \quad \Gamma \vdash f : \Pi n{:}\mathbb{N}.\, P\,n \to P\,(s\,n)}{\Gamma \vdash rec\,P\,z\,f : \Pi n{:}\mathbb{N}.\, P\,n}\ \text{(elim)}$$

To get $P\,n$ for every n: give it for 0, and turn it for n into it for $s\,n$.

Instance

$$\frac{\Gamma \vdash 1 : \mathbb{N} \qquad \Gamma \vdash \lambda n{:}\mathbb{N}.\, \lambda r{:}\mathbb{N}.\, s\,n \cdot r : \mathbb{N} \to \mathbb{N} \to \mathbb{N}}{\Gamma \vdash rec\,(\lambda n{:}\mathbb{N}.\, \mathbb{N})\,1\,(\lambda n\,r.\, s\,n \cdot r) : \mathbb{N} \to \mathbb{N}}\ P \equiv \lambda n{:}\mathbb{N}.\, \mathbb{N}$$

This is fac: r is the result for n.

Not an instance

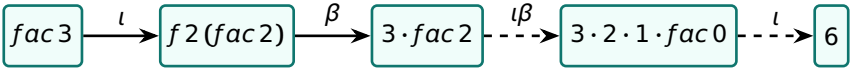
$f \equiv \lambda n{:}\mathbb{N}.\, s\,n$ its type is $\mathbb{N} \to \mathbb{N}$: the step must also take the result for n

N&G §6.5; TPIL §7.4

The rule (ι): the recursor computes

$$rec\,P\,z\,f\,0 \to_\iota z \qquad\qquad rec\,P\,z\,f\,(s\,n) \to_\iota f\,n\,(rec\,P\,z\,f\,n)$$

A recursor applied to a constructor reduces. With $fac \equiv rec\,P\,1\,f$ and $f \equiv \lambda n\,r.\,s\,n \cdot r$:



arithmetic is computed the same way

($conv$) now identifies terms up to three reductions: β, a λ meets an argument; δ, a definition unfolds; ι, a recursor meets a constructor. Confluence and normalisation still hold, so type checking stays decidable. N&G §16.2, §10.5

What `inductive` generates

```
inductive N where
  | zero : N
  | succ : N → N

#check @N.rec
-- @N.rec : {motive : N → Sort u_1} →
--   motive N.zero →
--   ((a : N) → motive a → motive a.succ) →
--   (t : N) → motive t

#print axioms N.rec   -- 'N.rec' does not depend on any axioms
```

`motive` is *P*; `Sort u_1` lets it land in `Prop` or in `Type`. The recursor has no body, yet the audit lists no axiom: it comes with the rule, it is not assumed.

The recursor computes

```
def fac' : Nat → Nat :=
  Nat.rec (motive := fun _ => Nat) 1 (fun n r => (n + 1) * r)

#reduce fac' 3                    -- 6
example : fac' 3 = 6 := rfl       -- (conv), no proof steps
#eval fac' 5                      -- 120
```

`rfl` is (*conv*): the kernel itself runs the ι-steps.

Pattern matching is the recursor

```
def fac : Nat → Nat
  | 0     => 1
  | n + 1 => (n + 1) * fac n

example : fac 3 = 6 := rfl
#eval fac 20                    -- 2432902008176640000

#print fac
-- def fac : Nat → Nat :=
-- fun x => Nat.brecOn x fac._f
```

The two equations are compiled into the recursor (`brecOn` is built from `Nat.rec`), so they compute in the same way.

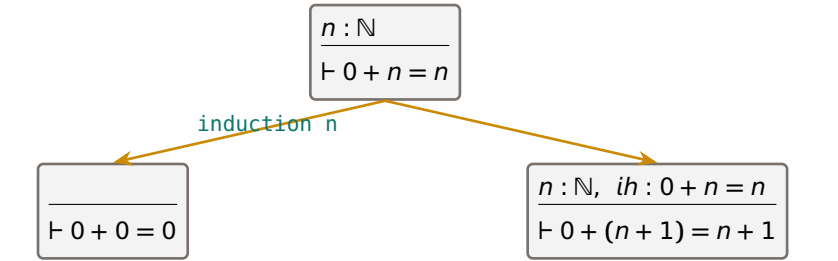
Induction is (*elim*) at a `Prop` motive

Take the motive into propositions, $P : \mathbb{N} \to *_p$, and read Π and $\to$ as $\forall$ and $\Rightarrow$:

$$\begin{aligned} rec\,P &: P\,0 \to (\Pi n{:}\mathbb{N}.\, P\,n \to P\,(s\,n)) \to \Pi n{:}\mathbb{N}.\, P\,n \\ ax\text{-}nat_3(P) &: (P\,0 \wedge \forall n{:}\mathbb{N}.\, (P\,n \Rightarrow P\,(s\,n))) \Rightarrow \forall n{:}\mathbb{N}.\, P\,n \end{aligned}$$

The same statement, up to currying the $\wedge$. The axiom is now an instance of a rule, and the two premisses are the base case and the step. N&G §14.1; TPIL §7.4

Induction as a goal state



the motive, read off the goal: $P \equiv \lambda n{:}\mathbb{N}.\ (0 + n = n)$

The step opens two flags, n and the induction hypothesis ih, over the goal $P\,(s\,n)$. The motive P is read off the goal.

TPIL §7.6

`induction`, and the term it built

```
theorem zero_add' (n : Nat) : 0 + n = n := by
  induction n with
  | zero      => rfl                      -- ⊢ 0 + 0 = 0
  | succ n ih => rw [Nat.add_succ, ih]    -- ⊢ 0 + (n + 1) = n + 1

#print zero_add'
-- theorem zero_add' : ∀ (n : Nat), 0 + n = n :=
-- fun n => Nat.recAux (Eq.refl (0 + 0)) (fun n ih => ⋯) n
```

The term applies the recursor to the two cases. `Nat.recAux` is `Nat.rec` with $s\,n$ written `n + 1`.

cases: elimination without the hypothesis

```
theorem zero_or_succ (n : Nat) : n = 0 ∨ ∃ m, n = m + 1 := by
  cases n with
  | zero   => exact Or.inl rfl
  | succ m => exact Or.inr ⟨m, rfl⟩
```

The lemma “every number is 0 or a successor” once more. `cases` is the recursor whose step ignores the induction hypothesis, as line (3) of the flag proof did.

Every inductive type has induction

```
theorem length_append' (xs ys : List Nat) :
    (xs ++ ys).length = xs.length + ys.length := by
  induction xs with
  | nil          => simp
  | cons x xs ih => simp [ih, Nat.add_right_comm]

open Clai.Lambda in
theorem size_pos (M : Term) : 0 < M.size := by
  induction M with
  | var _       => simp [Term.size]
  | app _ _ _ _ => simp [Term.size]
  | lam _ _ _   => simp [Term.size]
```

Lists have two constructors, two cases. `Term`, the λ-terms of the first lectures, has three: one per constructor.

Inductive predicates

A predicate defined by rules: the rules are its constructors, and a proof of $Even\,n$ is a derivation.

$$\frac{}{Even\,0}\ \text{(zero)} \qquad \frac{Even\,n}{Even\,(n+2)}\ \text{(add2)}$$

$$\dfrac{\dfrac{\dfrac{}{Even\,0}\ \text{zero}}{Even\,2}\ \text{add2}}{Even\,4}\ \text{add2} \quad \xrightarrow{\text{rule induction}} \quad \dfrac{\dfrac{\dfrac{}{P\,0}\ z}{P\,2}\ f}{P\,4}\ f$$

a derivation — the proof follows it

Rule induction: to prove $P\,n$ whenever $Even\,n$, prove P for the conclusion of each rule, assuming it for the premisses. N&G §5.1; TPIL §7.7

Rule induction, and inversion, in Lean

```
inductive Even : Nat → Prop where
  | zero : Even 0
  | add2 {n} : Even n → Even (n + 2)

theorem even_double (n : Nat) : Even (2 * n) := by
  induction n with
  | zero      => exact Even.zero
  | succ n ih => rw [Nat.mul_succ]; exact Even.add2 ih

theorem even_inv (n : Nat) (h : Even (n + 2)) : Even n := by
  cases h with
  | add2 h' => exact h'            -- ‘zero‘ cannot end in n + 2
```

cases on a derivation is **inversion**: only ($add2$) can conclude $Even\,(n+2)$, so its premiss $Even\,n$ is available.

The connectives and $=$ are inductive types

type	constructors: introduction	recursor: elimination
$A \wedge B$	$A \to B \to A \wedge B$	$(A \to B \to C) \to A \wedge B \to C$
$A \vee B$	$A \to A \vee B, \quad B \to A \vee B$	$(A \to C) \to (B \to C) \to A \vee B \to C$
$\bot$	none	$\bot \to C$
$\exists x{:}S.\, P\,x$	$\Pi x{:}S.\, P\,x \to \exists x{:}S.\, P\,x$	$(\Pi x{:}S.\, P\,x \to C) \to (\exists x{:}S.\, P\,x) \to C$
$x =_S y$	$refl : \Pi x{:}S.\, x =_S x$	substitution: $x =_S y \to Q\,x \to Q\,y$

In λD these were codings, and their rules were derived. Here the rules are given, and $\bot$ is the type with no constructor. TPIL §7.3, §7.7

The connectives and =, in Lean

```
#print And
-- structure And (a b : Prop) : Prop
-- constructor:
--   And.intro {a b : Prop} (left : a) (right : b) : a ∧ b

#check @Or.rec
-- ∀ {a b : Prop} {motive : a ∨ b → Prop},
--   (∀ (h : a), motive ⋯) → (∀ (h : b), motive ⋯) →
--   ∀ (t : a ∨ b), motive t

#print False   -- inductive False : Prop,  constructors: (none)

#print Eq
-- inductive Eq : {α : Sort u_1} → α → α → Prop
-- constructors: Eq.refl : ∀ {α : Sort u_1} (a : α), a = a
```

Three ways to recurse

Every term of λC has a normal form, which keeps type checking decidable. Lean keeps that by checking that each recursion terminates.

structural

`fac (n+1)` calls `fac n`

a call on a constructor's argument: the recursor

computes by `rfl`

well-founded

`gcd (x+1) y` calls `gcd (y % (x+1)) (x+1)`

a measure decreases; you prove it

runs, and is proved by its equations

partial

`collatz n` calls `collatz (3*n+1)`

no termination argument

runs; nothing about it is provable

N&G Thm. 6.3.14; TPIL §8.5

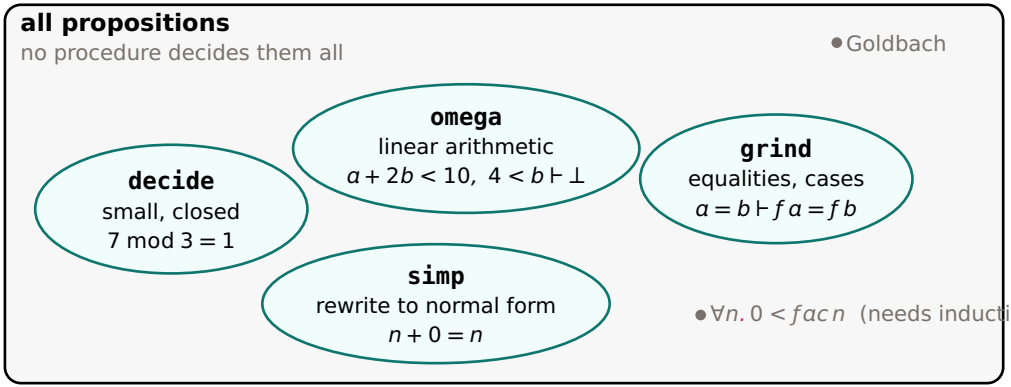
Termination is an obligation

```
def gcd' : Nat → Nat → Nat
  | 0,     y => y
  | x + 1, y => gcd' (y % (x + 1)) (x + 1)
termination_by x _ => x
decreasing_by exact Nat.mod_lt _ (Nat.succ_pos x)

#eval gcd' 12 18                                -- 6
#guard_msgs(drop error) in
example : gcd' 12 18 = 6 := rfl                 -- error: not by (conv)
example : gcd' 12 18 = 6 := by simp [gcd']      -- by its equations
```

`termination_by` names the measure, `decreasing_by` proves it drops. The function is not built by the recursor, so it does not reduce by ι, and `rfl` fails.

Automation decides fragments



Finding a proof is undecidable in general; inside a fragment, a procedure may decide it.

N&G Thm. 6.3.15

`decide` and `omega`

```
example : 7 % 3 = 1 := by decide
example : ∀ n < 5, n * n < 20 := by decide
#guard_msgs(drop error) in
example : ∀ n < 100000, n + 0 = n := by decide  -- too big

example (a b : Nat) (h : a + 2 * b < 10) (k : 4 < b) : False := by
  omega
example (n : Nat) : n ≤ 2 * n := by omega
#guard_msgs(drop error) in
example (a b : Nat) : 0 ≤ a * b := by omega       -- a * b: not linear
```

`decide` evaluates the proposition in the kernel, so it must be small. `omega` decides linear arithmetic over $\mathbb{N}$ and $\mathbb{Z}$: $+$, $<$, and multiplication by constants.

grind

```
example (f : Nat → Nat) (a b c : Nat) (h₁ : a = b) (h₂ : b = c) :
    f (f a) = f (f c) := by grind
example (xs ys : List Nat) :
    (xs ++ ys).length = xs.length + ys.length := by grind
theorem g (p q : Prop) (a b : Nat) (h : p ∨ q)
    (hp : p → a < b) (hq : q → a < b) : a ≤ b := by grind

#print axioms g
-- 'g' depends on axioms: [propext, Classical.choice, Quot.sound]
```

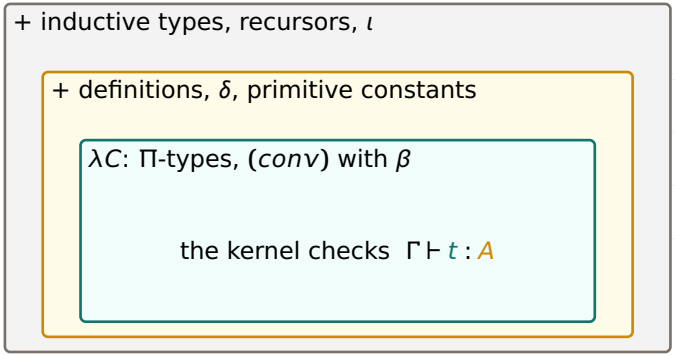
Equalities, case splits and arithmetic in one search. It leaves an ordinary term, which the kernel checks; no new axiom appears.

Which tactic for which goal

the goal is	try	because
two sides that compute	`rfl`, `decide`	the kernel runs $\beta\delta\iota$
an equation up to known lemmas	`simp`	rewriting to a normal form
linear arithmetic	`omega`	a decision procedure
equalities, case splits, arithmetic	`grind`	a proof-producing search
a statement about every n	`induction`	the recursor
anything else	a proof	no procedure covers it

The induction hypothesis, and the motive that makes it strong enough, are the first things no procedure supplies.

The kernel, assembled



untrusted: they only propose t

<- - - - elaborator

<- - - - tactics

<- - - - simp, omega, grind

<- - - - a language model

Elaborator, tactics and models only propose terms; the kernel checks them. Checking stays decidable; finding does not.

N&G Thm. 6.3.15, §16.2

What is in hand, and what comes next

propositions about	**stated with**	**proved by**
logic	the connectives (L4)	their rules, as terms or scripts
named notions	definitions (L5)	unfolding, and named lemmas
equality, sets	Eq, predicates (L6)	the motive, `rw`, `simp`
numbers, lists, derivations	inductive types (L7)	the recursor, `induction`

Next

A program is a term, its specification is a predicate on its inputs and outputs, and a proof that the program meets it is a term of that type. Nothing new is added to the theory.

References I

Avigad, J., L. de Moura, S. Kong, and S. Ullrich (2026). *Theorem Proving in Lean 4*. With contributions from the Lean community; tested against Lean 4.33.0. URL: https://lean-lang.org/theorem_proving_in_lean4/ (visited on 08/20/2026).

Church, A. (1936). “A note on the Entscheidungsproblem”. In: *Journal of Symbolic Logic* 1.1, pp. 40–41.

Coquand, T. and G. Huet (1988). “The calculus of constructions”. In: *Information and Computation* 76.2–3, pp. 95–120.

Girard, J.-Y. (1972). “Interprétation fonctionnelle et élimination des coupures de l’arithmétique d’ordre supérieur”. Thèse d’État. Université Paris VII.

Martin-Löf, P. (1984). *Intuitionistic Type Theory*. Naples: Bibliopolis.

Nederpelt, R. and H. Geuvers (2014a). “Definitions”. In: *Type Theory and Formal Proof: An Introduction*. Cambridge: Cambridge University Press. Chap. 8, pp. 165–188.

References II

Nederpelt, R. and H. Geuvers (2014b). “Further perspectives”. In: *Type Theory and Formal Proof: An Introduction*. Cambridge: Cambridge University Press. Chap. 16, pp. 379–390.

— (2014c). “Numbers and arithmetic in λD”. In: *Type Theory and Formal Proof: An Introduction*. Cambridge: Cambridge University Press. Chap. 14, pp. 305–348.

— (2014d). “The Calculus of Constructions”. In: *Type Theory and Formal Proof: An Introduction*. Cambridge: Cambridge University Press. Chap. 6, pp. 123–136.

— (2014e). *Type Theory and Formal Proof: An Introduction*. Cambridge: Cambridge University Press.

Peano, G. (1889). *Arithmetices principia, nova methodo exposita*. Turin: Bocca.

References III

Presburger, M. (1930). “Über die Vollständigkeit eines gewissen Systems der Arithmetik ganzer Zahlen, in welchem die Addition als einzige Operation hervortritt”. In: *Comptes Rendus du I Congrès des Mathématiciens des Pays Slaves, Warszawa 1929*, pp. 92–101.

Turing, A. M. (1936). “On computable numbers, with an application to the Entscheidungsproblem”. In: *Proceedings of the London Mathematical Society*. 2nd ser. 42, pp. 230–265.

§14.1 (Fig. 14.1, Lem. 14.1.4) for the Peano axioms as primitive definitions; §8.2 and §16.2 (pp. 383–384) for recursion as computation; §6.5 for CIC; Thm. 6.3.14, 6.3.15 and §10.5 for normalisation, confluence and the decidability boundary. The Lean rail follows *Theorem Proving in Lean 4*: ch. 7 (inductive types, §7.8 for what a declaration generates), ch. 8 (structural and well-founded recursion), §5.7 for `decide` and `omega`.